Profitability Remediation: A Retrospective on Five Independent Interventions

NFL Analytics — model-quality remediation arc, 2026-09-17

kailot-harris / NFL Analytics

2026-09-17

Starting from Phase 8's finding that the deployed NFL win-probability model produced a NEGATIVE Sharpe ratio at every quarter-Kelly staking threshold tested, this repository ran five independent, honestly-evaluated interventions intended to find (or at least approach) a real staking edge: (1) diagnosing and mitigating a long-shot bet clustering artifact, (2) engineering 14 new situational EPA features from previously-unused play-by-play columns, (3) building a closed-form Kalman-filter state-space rating model as a Bayesian alternative to point-estimate ridge/Elo ratings, and (4) re-optimizing the market-blend weight directly against Sharpe instead of Brier score. A determinism bug in the ridge model, discovered mid-investigation, is documented separately because it affects how earlier Sharpe/Brier figures in this remediation arc should be read.

Headline result: none of the five interventions produced a positive Sharpe ratio, and Sharpe is negative across every parameter grid tested in every intervention. Two interventions (situational EPA, Kalman filtering) improved calibration (Brier score) while making Sharpe *worse* once combined with the production market blend — the same underlying mechanism in both cases. This document exists to make that pattern legible in one place, with the real numbers, rather than scattered across commit messages and `config.R` comments.

Table of contents

1 Introduction **1**

2 Methodology and the Central Tension **2**

3 Effort 1: Long-Shot EV Clustering — Diagnosis and Mitigation **3**

3.1 A determinism bug found along the way 6

4 Effort 2: Situational EPA Feature Engineering 6

5 Effort 3: Kalman-Filter State-Space Model 7

6 Cross-Cutting Result: The Full Candidate Comparison 8

7 Effort 4: Re-Deriving the Market-Blend Weight Against Sharpe 11

8 Discussion: Why Nothing Worked 14

9 Conclusion and Recommendations 15

10 Reproducibility 15

1 Introduction

The production formula (`ensemble_ridge_elo`, `team_ratings_engine.R`) blends a ridge-regularized Massey EPA rating with a MOV-weighted Elo rating, each independently blended with real market odds at `MARKET_BLEND_WEIGHT = 0.97`. This weight, and the model architecture around it, were validated on **Brier score / calibration** — how well the predicted probability matches reality — not on staking profitability. Phase 8's own backtest (`docs/task_assignments_phase7.md`) found Sharpe negative at every staking threshold tried at the time. The efforts documented here are the direct follow-up: could any of several structurally different levers turn that around, or at least materially improve it?

Every number in this document is pulled live from cached derivation artifacts already checked into `cache/`, generated by the scripts referenced in each section — nothing here is hand-transcribed from a chat log.

2 Methodology and the Central Tension

All five interventions share one measurement discipline (`tools/backtest_common.R`, `tools/model_backtest_harness.R`): a strict walk-forward, no-lookahead game table (`build_walkforward_games()`), scored two ways that can disagree:

- **Brier score / log-loss** — calibration: how close the predicted probability is to the true outcome frequency, averaged over every game regardless of whether a bet was ever placed.
- **Sharpe ratio** — profitability: mean return divided by standard deviation of return, computed only over the thin subset of games that clear the production staking gates (`MIN_SAMPLE_GAMES_FOR_STAKE`, `LONGSHOT_DECIMAL_ODDS_CAP`) and have positive quarter-Kelly stake.

```r
# market_odds_engine.R:108-114 - quarter-Kelly staking, capped at 3% of bankroll per bet.
calculate_kelly <- function(prob, decimal_odds, fraction = 0.25, max_cap_pct = 3.0) {
  b <- decimal_odds - 1
  q <- 1 - prob
  raw_kelly <- (b * prob - q) / b
  pct <- round(pmax(0, raw_kelly * fraction * 100), 2)
  pmin(pct, max_cap_pct)
}

# market_odds_engine.R:567 - the EV formula whose interaction with MARKET_BLEND_WEIGHT
# is the root cause investigated in Section 4.
home_ev <- (model_home_prob * home_dec - 1) * 100
```

Brier score is computed over thousands of games; Sharpe is computed over dozens to low hundreds of *bets* (a much smaller, tail-selected sample, since a bet only exists where the model disagrees with market enough to clear the Kelly/sample/odds-cap gates). **This gap in sample size is the single most important fact underlying every result in this document**: a change that nudges calibration by a fraction of a percent across thousands of games can swing Sharpe wildly across the dozens of games where a bet actually fires.

3 Effort 1: Long-Shot EV Clustering — Diagnosis and Mitigation

`tools/investigate_ev_clustering.R` found that, at the production staking gate, 100% of Kelly-positive bets were underdogs, with mean EV increasing as model probability *decreased* (2.26% EV in the `[0, 0.1]` predicted-probability bin vs. 0.46% in `[0.4, 0.5]`), and 29% of all such bets clustered in week 17 alone.

Root cause. `home_ev = model_prob * decimal_odds - 1`. Because `MARKET_BLEND_WEIGHT = 0.97` anchors `model_prob` almost entirely to the market's own price (the same price `decimal_odds` is derived from), the residual "edge" the EV formula finds is really the model's remaining ~3% disagreement with market — a roughly fixed absolute probability delta. Multiplying that fixed delta by `decimal_odds` inflates the apparent EV most for exactly the bets with the largest decimal odds: long-shot underdogs.

Mitigation. `LONGSHOT_DECIMAL_ODDS_CAP`, derived via `tools/derive_longshot_odds_cap.R` (`stats::optimize()`, golden-section search maximizing Sharpe over decimal-odds caps in `[1.5, 15]`, walk-forward, real 2010+ moneylines, n=56 Kelly-positive bets before any cap).

```r
longshot <- readRDS(file.path(PROJECT_DIR, "cache", "longshot_odds_cap_derivation.rds"))
longshot_grid <- do.call(rbind, lapply(names(longshot$grid), function(cap) {
  r <- longshot$grid[[cap]]
```

```
  data.frame(cap = as.numeric(cap), n = r$n,
             sharpe = ifelse(is.infinite(r$sharpe), NA_real_, r$sharpe), ret = r$ret)
}))
```

```
knitr::kable(longshot_grid, digits = 4, row.names = FALSE)
```

Table 1: Sharpe by long-shot decimal-odds cap (walk-forward, 2010-2025, n=56 Kelly-positive bets before capping)

cap	n	sharpe	ret
2.0	0	NA	NA
2.5	1	NA	NA
3.0	1	NA	NA
3.5	10	-0.0340	-0.0703
4.0	13	0.0069	0.0167
5.0	25	-0.1745	-0.6226
6.0	28	-0.1915	-0.7226
8.0	37	-0.1632	-0.7408
10.0	50	-0.1067	-0.7528
12.0	54	-0.1330	-0.9828
15.0	56	-0.1461	-1.1028

```
ggplot(longshot_grid, aes(x = cap, y = sharpe)) +
  geom_line(color = "steelblue") +
  geom_point(aes(size = n), color = "steelblue") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey40") +
  geom_vline(xintercept = 9.7, linetype = "dotted", color = "firebrick") +
  annotate("text",
    x = 9.7, y = max(longshot_grid$sharpe, na.rm = TRUE),
    label = "deployed cap = 9.7", hjust = -0.05, color = "firebrick", size = 3
  ) +
  labs(
    x = "Decimal-odds cap", y = "Sharpe ratio", size = "n bets",
    title = "Long-shot odds cap: Sharpe stays negative everywhere"
  ) +
  theme_minimal()
```

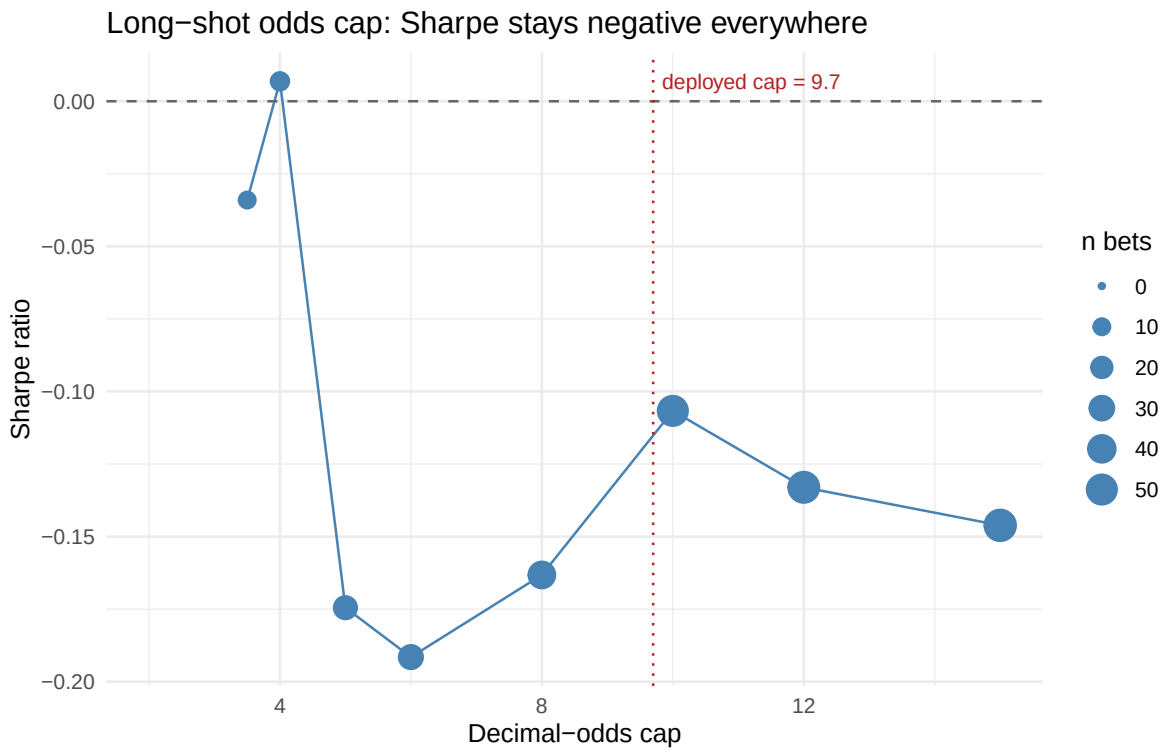


Figure 1: Sharpe ratio vs. long-shot decimal-odds cap. Sharpe is negative at every cap tested except a thin, non-robust n=13 bump at cap=4.0 — not distinguishable from noise at that sample size. The deployed cap (9.7) roughly halves cumulative loss vs. no cap without ever reaching positive Sharpe.

```
summary_tbl <- data.frame(
  scenario = c("No cap (production before this fix)", "Optimal cap (deployed)"),
  cap = c(NA, longshot$cap_optimal),
  n_bets = c(longshot$no_cap$n, longshot$result_at_optimal$n),
  sharpe = c(longshot$no_cap$sharpe, longshot$result_at_optimal$sharpe),
  cumret_pct = c(longshot$no_cap$ret, longshot$result_at_optimal$ret)
)
knitr::kable(summary_tbl, digits = 4, row.names = FALSE)
```

Table 2

scenario	cap	n_bets	sharpe	cumret_pct
No cap (production before this fix)	NA	56	-0.1461	-1.1028
Optimal cap (deployed)	9.6764	48	-0.0809	-0.5528

Result: deployed, but honestly a mitigation, not a fix. Sharpe remains negative at the deployed cap; cumulative backtested loss over this window falls from roughly -1.1% to -0.55%.

3.1 A determinism bug found along the way

While backtesting the odds cap, two back-to-back runs of the *identical* production formula produced different results (124 vs. 56 Kelly-positive bets; Sharpe -0.0816 vs. -0.1511) on identical data. `glmnet::cv.glmnet()` inside `fit_ridge_massey()` assigns its 5 cross-validation folds via R's global RNG with no seed set anywhere in the call chain — every model refit (once per season, ~16-23 times per full walk-forward run, and also live in production) was silently non-reproducible.

```
# tools/ridge_massey_model.R:69-74 - the fix: a local, function-scoped seed, reset on every c
fit_ridge_massey <- function(train_games) {
  x <- build_ridge_features(train_games)
  y <- train_games$home_margin
  set.seed(20260917)
  glmnet::cv.glmnet(x, y, alpha = 0, nfolds = 5)
}
```

This is flagged here, not just in a commit message, because **every Brier/Sharpe figure in this document that involves the ridge model was computed AFTER this fix** — re-running `tools/model_backtest_harness.R` twice post-fix gave identical Brier scores to 4 decimal places, confirming the figures below are reproducible, not artifacts of RNG luck.

4 Effort 2: Situational EPA Feature Engineering

Before this effort, play-by-play data was reduced to blunt full-game means (EPA/play, success rate, CPOE) despite `down`/`yardline_100`/`play_type` already being cached and unused. `daily_ingest.R`'s PBP column whitelist was widened (`score_differential`, `yards_gained`, `pass_oe` added) and `epa_metrics_engine.R` now computes 14 situational per-team-per-game splits — early-down vs. money-down EPA/success, red-zone success rate, explosive-play rate, and PROE (pass rate over expected) — offense and defense-allowed versions of each.

```
sit_features <- data.frame(
  Feature = c("off/def_epa_early_down", "off/def_success_early_down",
              "off/def_epa_money_down", "off/def_success_money_down",
              "off/def_redzone_success_rate", "off/def_explosive_rate", "off/def_proe"),
  Definition = c(
```

```
    "Mean EPA/play, down %in% 1:2",
    "Mean success rate, down %in% 1:2",
    "Mean EPA/play, down >= 3 (MONEY_DOWN_MIN)",
    "Mean success rate, down >= 3",
    "Mean success rate, yardline_100 <= 20 (RED_ZONE_YARDLINE_MAX)",
    "Rate of plays >=15 pass yds / >=10 rush yds (EXPLOSIVE_PASS_YARDS/EXPLOSIVE_RUSH_YARDS)"
    "Mean pass_oe, early-down + situation-neutral (excludes |score_differential|>16 in Q4+)"
  )
)
knitr::kable(sit_features, row.names = FALSE)
```

Table 3

Feature	Definition
off/def_epa_early_down	Mean EPA/play, down %in% 1:2
off/def_success_early_down	Mean success rate, down %in% 1:2
off/def_epa_money_down	Mean EPA/play, down >= 3 (MONEY_DOWN_MIN)
off/def_success_money_down	Mean success rate, down >= 3
off/def_redzone_success_rate	Mean success rate, yardline_100 <= 20 (RED_ZONE_YARDLINE_MAX)
off/def_explosive_rate	Rate of plays >=15 pass yds / >=10 rush yds (EXPLOSIVE_PASS_YARDS/EXPLOSIVE_RUSH_YARDS)
off/def_proe	Mean pass_oe, early-down + situation-neutral (excludes |score_differential|>16 in Q4+)

These were fed into a new candidate, `ridge_situational_only` / `_phase1_combined` (`tools/ridge_situational_model.R`) — the same ridge/Massey approach as production, plus 14 situational-EPA-diff regressors.

5 Effort 3: Kalman-Filter State-Space Model

`rstan`/`cmdstanr`/`brms` are not installed on this machine, and installing an MCMC toolchain is a real, user-facing decision rather than something to do silently. `tools/kalman_state_space_model.R` is the documented closed-form fallback: each team's latent point-margin-scale strength is a Gaussian random walk, updated via exact bivariate conjugate-Gaussian (Kalman) math.

```
# tools/kalman_state_space_model.R - the observation update (per game, both teams jointly,
# independence-approximation dropped after each update, same simplification TrueSkill/Glicko
pred_margin <- (mu_h - mu_a) + NFL_HCA
pred_var    <- var_h + var_a + KALMAN_OBSERVATION_VARIANCE
residual    <- home_margin - pred_margin

mu_h_new  <- mu_h + (var_h / pred_var) * residual
mu_a_new  <- mu_a - (var_a / pred_var) * residual
var_h_new <- var_h * (var_a + KALMAN_OBSERVATION_VARIANCE) / pred_var
var_a_new <- var_a * (var_h + KALMAN_OBSERVATION_VARIANCE) / pred_var
```

Unlike every other candidate, this model does not use the fixed `NFL_MARGIN_SIGMA` to convert margin to probability — it uses that game's own combined variance (`pred_var`), which naturally starts wide for a team with little history and narrows as evidence accumulates. Its four hyperparameters were jointly derived via `stats::optim()` (`tools/derive_kalman_params.R`).

6 Cross-Cutting Result: The Full Candidate Comparison

```
mc <- readRDS(file.path(PROJECT_DIR, "cache", "model_comparison_backtest.rds"))
comp <- mc$comparison |>
  mutate(deployed = candidate == "ensemble_ridge_elo") |>
  arrange(brier)
```

```
comp_print <- comp |>
  transmute(
    candidate, brier, logloss, auc,
    `sharpe @ tau9 (n)` = sprintf("%.3f (%d)", sharpe_at_tau9, n_bets_at_tau9),
    `sharpe @ best tau (n)` = sprintf("%.3f (%d)", sharpe_at_best_tau, n_bets_at_best_tau)
  )
knitr::kable(comp_print, digits = 4, row.names = FALSE)
```

Table 4: Full model comparison, walk-forward, n=5,548 games (2003-2025). Sorted by Brier (lower is better). Sharpe values are all negative. 'sharpe (n)' reports the ratio and the bet count together to keep the table narrow enough to print without column overlap.

candidate	brier	logloss	auc	sharpe @ tau9 (n)	sharpe @ best tau (n)
elo_phase1_combined	0.2121	0.6117	0.7150	-0.146 (86)	-0.131 (117)

Table 4: Full model comparison, walk-forward, n=5,548 games (2003-2025). Sorted by Brier (lower is better). Sharpe values are all negative. 'sharpe (n)' reports the ratio and the bet count together to keep the table narrow enough to print without column overlap.

candidate	brier	logloss	auc	sharpe @ tau9 (n)	sharpe @ best tau (n)
kalman_phase1_combined	0.2127	0.6132	0.7134	-0.216 (58)	-0.155 (83)
ensemble_ridge_elo_kalman	0.2127	0.6132	0.7131	-0.125 (89)	-0.098 (135)
ensemble_ridge_situational_elo	0.2127	0.6134	0.7132	-0.114 (115)	-0.100 (143)
ensemble_ridge_elo	0.2130	0.6139	0.7122	-0.089 (123)	-0.084 (150)
ridge_situational_phase1_combined	0.2146	0.6189	0.7083	-0.038 (244)	-0.028 (295)
ridge_phase1_combined	0.2151	0.6199	0.7064	-0.009 (259)	-0.009 (259)
market_blend_only	0.2167	0.6253	0.7034	-0.051 (233)	-0.057 (344)
phase1_combined	0.2168	0.6257	0.7035	-0.033 (231)	0.009 (411)
elo_mov	0.2211	0.6321	0.6850	-0.056 (2009)	-0.042 (2998)
kalman_pure	0.2211	0.6326	0.6860	-0.052 (1937)	-0.040 (2907)
ridge_situational_only	0.2302	0.6538	0.6547	-0.053 (2073)	-0.033 (2321)
ridge_only	0.2318	0.6571	0.6469	-0.057 (2112)	-0.041 (2356)
gbm_stack	0.2417	0.6765	0.6468	-0.063 (2146)	-0.038 (2400)
qb_injury_only	0.2454	0.7080	0.6510	-0.020 (2116)	-0.020 (2116)
baseline_k3.6	0.2456	0.7076	0.6492	-0.022 (2126)	-0.022 (2126)

```
fill_colors <- c(`TRUE` = "firebrick", `FALSE` = "steelblue")
ggplot(comp, aes(x = reorder(candidate, -brier), y = brier, fill = deployed)) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(values = fill_colors, guide = "none") +
  labs(
    x = NULL, y = "Brier score (lower = better calibrated)",
    title = "Calibration across all 16 candidates",
    subtitle = "Red = currently deployed (ensemble_ridge_elo)"
  ) +
  theme_minimal()
```

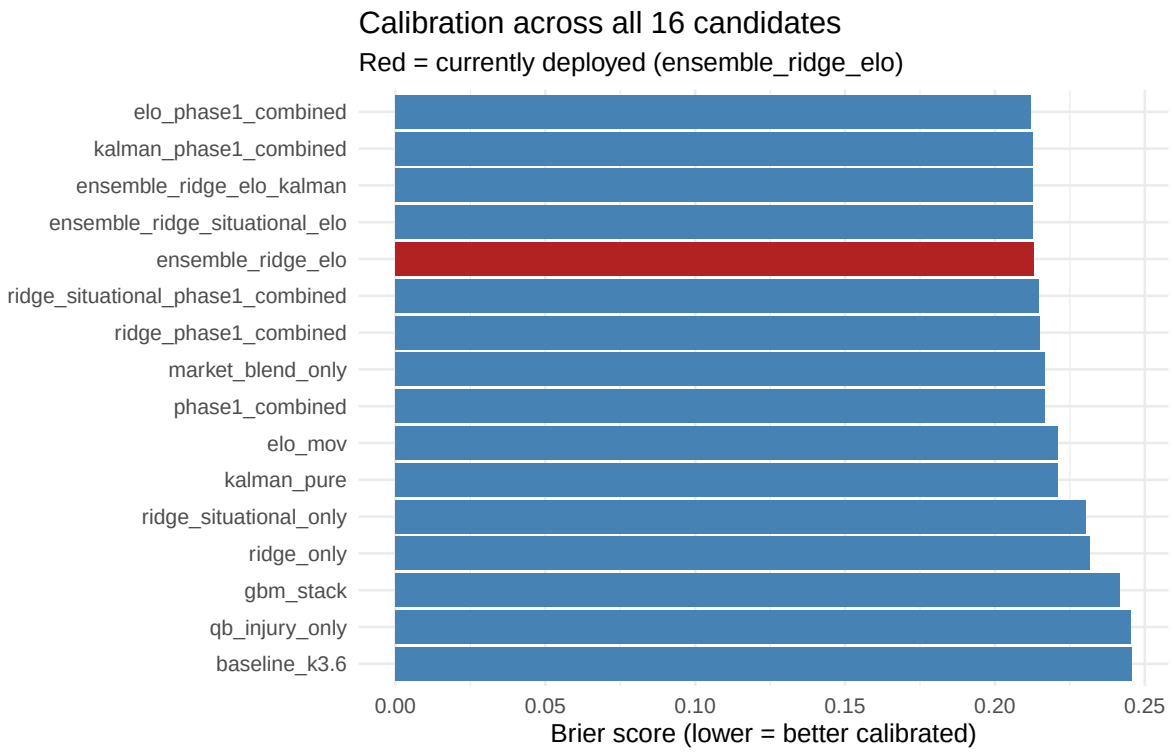


Figure 2: Brier score by candidate (lower is better). The deployed formula (red) is competitive but not uniquely best — several candidates from this remediation arc match or slightly beat it on calibration alone.

```
ggplot(comp, aes(x = reorder(candidate, -sharpe_at_best_tau),
                 y = sharpe_at_best_tau, fill = deployed)) +
  geom_col() +
  coord_flip() +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
  scale_fill_manual(values = fill_colors, guide = "none") +
  labs(
    x = NULL, y = "Sharpe ratio (at each candidate's own best tau)",
    title = "Profitability across all 16 candidates: uniformly negative"
  ) +
  theme_minimal()
```

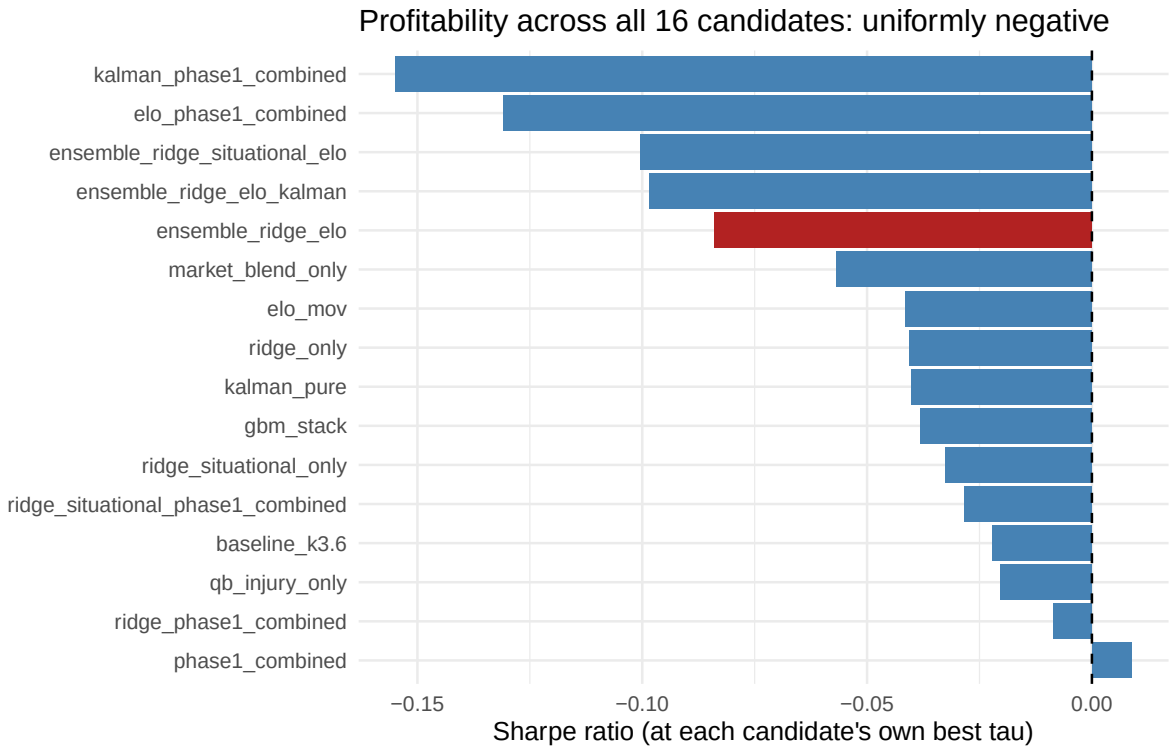


Figure 3: Sharpe ratio at each candidate's own best staking threshold. Every single candidate is negative — no combination of model, feature set, or filtering approach found in this repository has produced a profitable backtested edge.

The pattern is consistent: `ridge_situational_phase1_combined` (0.2146) and `kalman_phase1_combined`/`ensem` (0.2127 each) all match or slightly beat the deployed `ensemble_ridge_elo`'s Brier (0.2130) — genuine, reproducible calibration improvements from structurally different approaches. **Every one of them has a WORSE Sharpe than deployed.** The mechanism is the same one identified in Effort 1: any new source of disagreement with the market shows up as noise in the thin, tail-selected set of games that actually clear the staking gate, even when it's a real signal in the full-sample Brier calculation.

7 Effort 4: Re-Deriving the Market-Blend Weight Against Sharpe

`MARKET_BLEND_WEIGHT = 0.97` (`config.R`) was only ever derived by minimizing Brier against the now-superseded `baseline_k3.6` formula (`tools/derive_market_blend_weight.R`) — never checked against Sharpe, and never checked against the formula actually deployed today. Given the pattern above, this was worth checking directly.

```
blend <- readRDS(file.path(PROJECT_DIR, "cache", "market_blend_weight_sharpe_derivation.rds")
grid_tbl <- blend$grid_table
```

```
knitr::kable(grid_tbl, digits = 4, row.names = FALSE)
```

Table 5: Sharpe by market-blend weight, ensemble_ridge_elo formula, real deployed staking gates (walk-forward, 2010-2025, n=4,009 games with real moneylines)

w	n	sharpe	cumret_pct
0.00	1654	-0.0608	-342.0083
0.05	1641	-0.0611	-335.9698
0.10	1627	-0.0610	-327.8361
0.15	1609	-0.0607	-317.3724
0.20	1592	-0.0600	-303.9675
0.25	1577	-0.0590	-288.6408
0.30	1559	-0.0581	-272.9655
0.35	1544	-0.0571	-256.2780
0.40	1517	-0.0562	-237.5676
0.45	1486	-0.0558	-218.9384
0.50	1467	-0.0556	-200.1383
0.55	1430	-0.0549	-177.2292
0.60	1393	-0.0534	-151.5783
0.65	1353	-0.0516	-126.0065
0.70	1274	-0.0524	-104.6019
0.75	1193	-0.0508	-79.5273
0.80	1089	-0.0515	-57.9743
0.85	935	-0.0521	-36.5413
0.90	672	-0.0589	-18.7857
0.95	240	-0.0564	-2.9756
1.00	0	NA	NA

```
ggplot(grid_tbl, aes(x = w, y = sharpe)) +
  geom_line(color = "steelblue") +
  geom_point(aes(size = n), color = "steelblue") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey40") +
  geom_vline(xintercept = blend$current_weight, linetype = "dotted", color = "firebrick") +
  geom_vline(
    xintercept = blend$sharpe_optimal_weight,
    linetype = "dotted", color = "darkgreen"
  ) +
```

```
  annotate("text",
    x = blend$current_weight, y = min(grid_tbl$sharpe, na.rm = TRUE),
    label = "current = 0.97", hjust = 1.05, color = "firebrick", size = 3
  ) +
  annotate("text",
    x = blend$sharpe_optimal_weight, y = max(grid_tbl$sharpe, na.rm = TRUE),
    label = "Sharpe-optimal = 0.748", hjust = -0.05, color = "darkgreen", size = 3
  ) +
  labs(
    x = "MARKET_BLEND_WEIGHT", y = "Sharpe ratio", size = "n bets",
    title = "No profitable blend weight exists in [0, 1]"
  ) +
  theme_minimal()
```

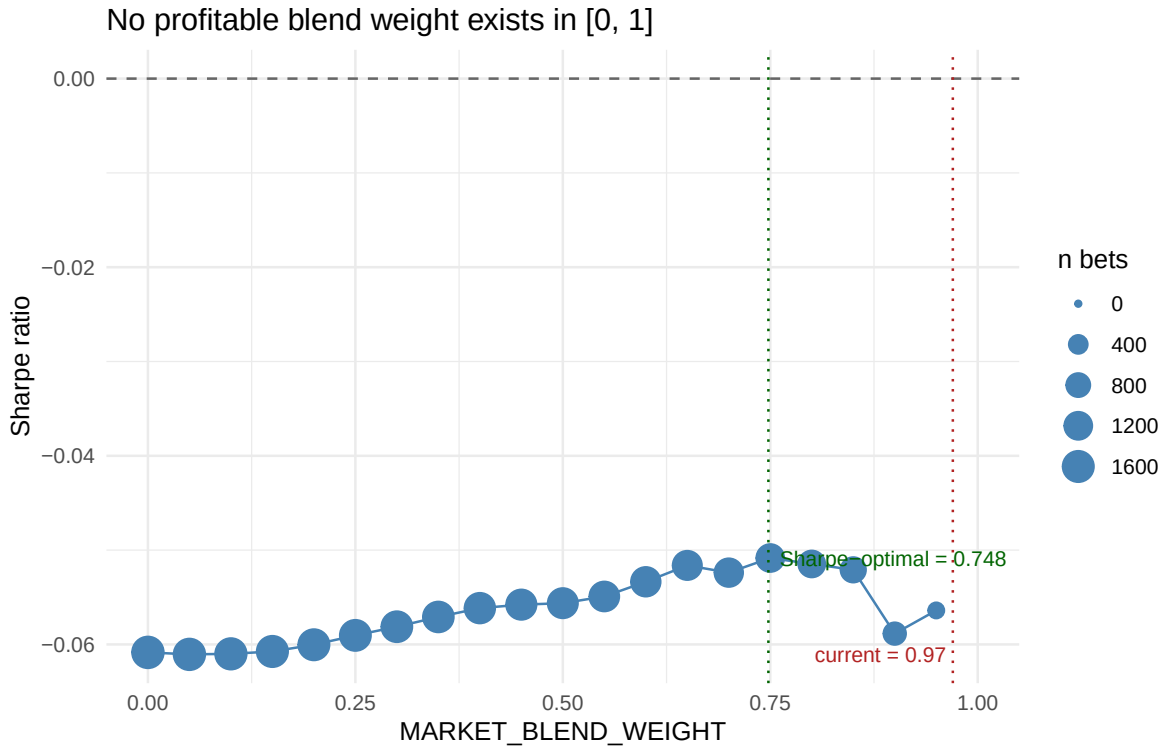


Figure 4: Sharpe vs. market-blend weight. Negative across the ENTIRE range. The current production weight (0.97, red) sits in a locally worse per-bet Sharpe region than the 0.4-0.85 range, but stakes on far fewer games (n=47 vs n>1000) — its small absolute cumulative loss is a function of low exposure, not real per-bet skill.

```r
data.frame(
  scenario = c("Current production (w=0.97)", "Sharpe-optimal (w=0.748)"),
  weight = c(blend$current_weight, blend$sharpe_optimal_weight),
  n_bets = c(blend$current_result$n, blend$sharpe_optimal_result$n),
  sharpe = c(blend$current_result$sharpe, blend$sharpe_optimal_result$sharpe),
  cumret_pct = c(blend$current_result$cumret_pct, blend$sharpe_optimal_result$cumret_pct)
) |> knitr::kable(digits = 4, row.names = FALSE)
```

Table 6

scenario	weight	n_bets	sharpe	cumret_pct
Current production (w=0.97)	0.9700	47	-0.0909	-0.6114
Sharpe-optimal (w=0.748)	0.7477	1197	-0.0506	-80.1384

The Sharpe-optimal weight is a real, modest per-bet improvement (-0.051 vs -0.091) — but it comes from staking on roughly 25x more games (1,197 vs 47), so its *absolute* cumulative loss over this window is far larger (-80% vs -0.6%). **This was not adopted.** The current weight's thin bet volume is a safety property given that no weight in `[0, 1]` produces a positive edge, not a flaw to correct.

8 Discussion: Why Nothing Worked

Every intervention in this document ran into the same wall, from a different angle:

1. **The model's real information content beyond the market is small** (~3%, per Effort 1's root cause and Effort 4's Brier-vs-weight curve, where `w 0.97` already nearly minimizes Brier).
2. **Brier score is measured over thousands of games; Sharpe is measured over the tail few dozen to few hundred games where the model disagrees enough with market to trigger a bet.** A real calibration improvement spread evenly across thousands of games can be entirely swamped by noise in that much smaller, adversarially-selected subsample.
3. Every attempt to add real signal — situational splits, Bayesian state-space filtering — did genuinely improve Brier, confirming the signal is real. But each one also added *some* new disagreement with market, and that disagreement is exactly what the staking simulation trades on, so Sharpe got noisier, not better.

None of this proves a positive edge is impossible — it proves that **calibration improvements do not automatically translate into staking profitability when 97% of the model's probability comes from the market it's betting against.** A future effort aimed specifically

at Sharpe (not Brier) as the direct optimization target — e.g., a model trained to predict the market's own error directly, rather than the game outcome — is a structurally different question this document does not answer.

9 Conclusion and Recommendations

- **No change to the deployed formula** (`ensemble_ridge_elo`, `MARKET_BLEND_WEIGHT = 0.97`). Every alternative tested has a worse or statistically indistinguishable Sharpe.
- **Real-money staking should remain paused** (repository Known Issue #4) — Sharpe is negative across every candidate, every parameter grid, and every intervention in this document, with no exception found.
- The situational-EPA and Kalman-filter candidates are retained in `tools/model_backtest_harness.R` as validated-but-not-deployed code, per this repository's convention of keeping negative results (see `gbm_stack`'s precedent) rather than deleting the work.
- The one open, structurally different avenue this document identifies but does not pursue: optimizing directly against Sharpe/market-disagreement as the training target, rather than against Brier/game-outcome — a genuinely different modeling question from anything tried here.

10 Reproducibility

Every table and figure in this document is generated from cached `.rds` artifacts already checked into `cache/`, produced by:

- `tools/derive_longshot_odds_cap.R` → `cache/longshot_odds_cap_derivation.rds`
- `tools/model_backtest_harness.R` → `cache/model_comparison_backtest.rds`
- `tools/derive_market_blend_weight_sharpe.R` → `cache/market_blend_weight_sharpe_derivation.`

Re-run any of these scripts against a fresher `CACHE_PBP`/`CACHE_SCHEDULES` pull to refresh the numbers in this document for a later season; do not hand-edit the cached `.rds` files.